1. What is Big Data and What is PySpark

What is Big Data?

Big Data refers to extremely large and complex datasets that cannot be processed, stored, or analyzed using traditional database management systems or data processing tools. The term is not just a

bout volume — it encompasses multiple dimensions often described by the Five Vs.

The 5 Vs of Big Data:

VARIETY	Structured, semi-structured, unstructured data
VERACITY	Accuracy, quality and trustworthiness of data
VELOCITY	High-speed streaming (IoT, social media, logs)
VALUE	Extracting meaningful insights from the data
VOLUME	Terabytes to petabytes of data generated daily

Real-World Examples of Big Data

- Facebook generates ~4 petabytes of data every single day
- Google processes ~8.5 billion search queries per day
- A single Boeing 737 generates ~240 terabytes of flight data per flight
- Amazon processes millions of transactions and clickstream events per second

Why Traditional Systems Fail

Traditional RDBMS (like MySQL or Oracle) process data on a single machine. When data exceeds the RAM or disk of that machine, they crash or become extremely slow. Big Data tools like Spark solve this by distributing the work across hundreds or thousands of machines.

Aspect	Traditional System	Big Data System (Spark)
Processing	Single machine	Distributed across cluster
Scalability	Vertical (add RAM/CPU)	Horizontal (add nodes)
Fault Tolerance	No automatic recovery	Built-in redundancy
Data Types	Structured only	Structured + unstructured

Speed	Batch-only, slow	Real-time + batch
Cost	Expensive hardware	Commodity hardware

What is Apache Spark?

Apache Spark is an open-source, unified analytics engine designed for large-scale data processing. It was developed at UC Berkeley's AMPLab in 2009 and later donated to the Apache Software Foundation. Spark's key innovation is its in-memory processing model, which makes it up to 100x faster than Hadoop MapReduce for certain workloads.

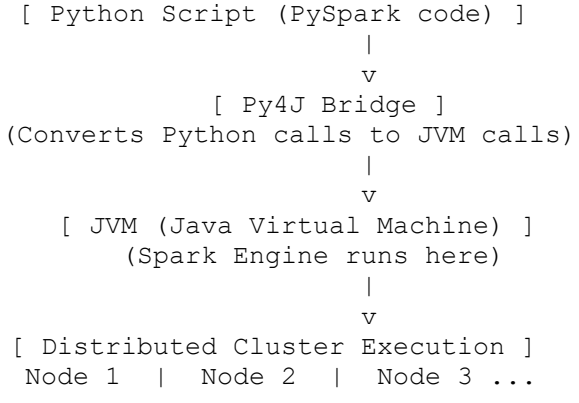
Core Features of Apache Spark

- Speed: In-memory processing with DAG (Directed Acyclic Graph) execution engine
- Ease of Use: High-level APIs in Python, Scala, Java, R and SQL
- Generality: Supports batch, streaming, ML, and graph processing in one platform
- Fault Tolerance: Automatic recovery using RDD lineage and checkpointing
- Compatibility: Runs on Hadoop YARN, Kubernetes, Mesos, or standalone

What is PySpark?

PySpark is the Python API (Application Programming Interface) for Apache Spark. It allows Python developers to write Spark programs without needing to know Scala or Java. Under the hood, PySpark uses Py4J, a library that bridges Python and the Java Virtual Machine (JVM) where Spark actually runs.

How PySpark Works Internally



PySpark Ecosystem Components

Component	Purpose	Example Use Case

Spark Core	Base engine, RDDs, scheduling	Basic data transformations
Spark SQL	SQL queries on structured data	Querying data with SELECT
Spark Streaming	Real-time data processing	Processing Kafka streams
MLlib	Machine learning library	Classification, clustering
GraphX	Graph computation	Social network analysis

2. Spark's Basic Architecture

Core Concepts: RDD, DAG, Transformations and Actions

Resilient Distributed Dataset (RDD)

An RDD is the fundamental data structure of Spark. It is an immutable, distributed collection of objects that can be processed in parallel across a cluster. The word 'Resilient' means it can automatically recover from node failures using lineage information.

- Immutable: Once created, an RDD cannot be changed
- Distributed: Data is split across multiple partitions on different nodes
- Lazy Evaluation: Transformations are not executed until an action is called
- Fault Tolerant: Lost partitions are recomputed from lineage graph

DAG (Directed Acyclic Graph)

When you write a Spark program, Spark does not execute operations immediately. Instead, it builds a DAG — a logical plan of all transformations. When an action is triggered, the DAG Scheduler converts this into physical stages and tasks, optimizes execution, and submits them to executors.

DAG Execution Flow

```
User Code: rdd.filter(...).map(...).groupBy(...).count()

          Step 1: DAG is built (logical plan)
      [filter] --> [map] --> [groupBy] --> [count]

         Step 2: DAG Scheduler splits into Stages
    Stage 1: [filter] --> [map]   (narrow dependency)
           SHUFFLE (data moves between nodes)
     Stage 2: [groupBy] --> [count] (wide dependency)

     Step 3: Task Scheduler sends Tasks to Executors
          Executor 1: Stage1-Task1, Stage2-Task1
          Executor 2: Stage1-Task2, Stage2-Task2
```

Transformations vs Actions

Type	Definition	Examples	When Executed
Transformation	Creates a new RDD/DataFrame from existing one	map, filter, groupBy, join, select	Lazy — only builds DAG

Action	Triggers computation, returns result to driver	count, collect, show, save, write	Immediately executes DAG

```
# Example of Transformations (Lazy) and Actions (Eager)
from pyspark.sql import SparkSession
spark = SparkSession.builder.master("local[*]").appName("DAG
Demo").getOrCreate()

# Load data - no computation yet
df = spark.read.csv("sales.csv", header=True, inferSchema=True)

# Transformations (lazy - just builds DAG)
filtered = df.filter(df["amount"] > 1000)          # Transformation
grouped  = filtered.groupBy("region")              # Transformation
result   = grouped.agg({"amount": "sum"})           # Transformation

# Action - NOW the DAG is executed!
result.show()   # Triggers computation of all steps above
```

3. Cluster Manager, Spark Session, Spark Context, Driver Node & Worker Node

Cluster Manager

The Cluster Manager is the external resource management system that Spark uses to acquire computing resources (CPU cores and memory) for running Spark applications. It acts as the intermediary between Spark and the physical machines in the cluster.

Analogy

Think of the Cluster Manager as an HR department. When Spark (the company) needs workers for a project, it asks HR (Cluster Manager) to assign people (executors) from available staff (worker nodes).

Types of Cluster Managers

Cluster Manager	Type	Best For	Key Feature
Standalone	Built-in Spark	Development & testing	Simple setup, no external dependencies
Apache YARN	Hadoop ecosystem	Enterprise Hadoop clusters	Integrates with HDFS, mature & widely used
Apache Mesos	General purpose	Mixed workloads	Fine-grained resource sharing
Kubernetes	Container-based	Cloud-native deployments	Docker containers, auto-scaling
Local Mode	Single machine	Development only	local[*] uses all CPU cores

SparkContext

SparkContext is the original entry point to Spark functionality (used in older Spark 1.x versions and still relevant for RDD operations). It represents a connection to a Spark cluster and can be used to create RDDs, accumulators, and broadcast variables. Every Spark application has exactly ONE SparkContext.

```
# Creating a SparkContext (older Spark 1.x style)
from pyspark import SparkContext, SparkConf

# Configure Spark
```

```
conf = SparkConf() \
    .setAppName("My App") \
    .setMaster("yarn") \
    .set("spark.executor.memory", "4g") \
    .set("spark.executor.cores", "2")

# Create SparkContext - ONE per application
sc = SparkContext(conf=conf)

# Create an RDD
data = [1, 2, 3, 4, 5]
rdd = sc.parallelize(data, numSlices=3)  # 3 partitions

# Apply transformation and action
result = rdd.map(lambda x: x * x).filter(lambda x: x > 5).collect()
print(result)  # [9, 16, 25]

# Always stop when done
sc.stop()
```

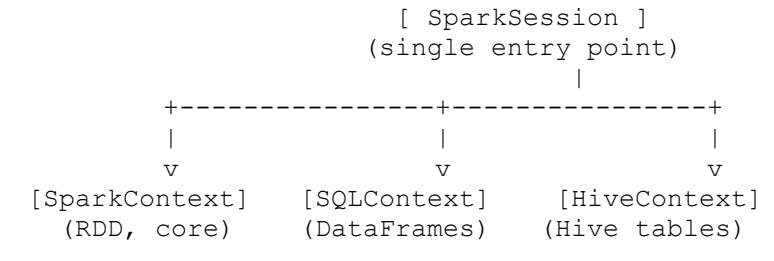
SparkContext Internals

- Connects to Cluster Manager to acquire resources
- Launches Executors on Worker Nodes
- Sends application code (JAR/Python files) to Executors
- Splits data into partitions and distributes across Executors
- Coordinates task scheduling via DAGScheduler and TaskScheduler

SparkSession

SparkSession was introduced in Spark 2.0 as a unified entry point that combines SparkContext, SQLContext, HiveContext, and StreamingContext into a single object. It is the recommended way to interact with Spark in modern applications. Internally, SparkSession creates a SparkContext.

SparkSession as a Unified API




```
# Creating SparkSession (modern Spark 2.x+ style)
```

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Sales Analysis") \
    .master("yarn") \
    .config("spark.executor.memory", "4g") \
    .config("spark.executor.cores", "2") \
    .config("spark.sql.shuffle.partitions", "200") \
    .enableHiveSupport() \   # Optional: enables Hive metastore
    .getOrCreate()            # Returns existing or creates new session

# Access SparkContext from SparkSession
sc = spark.sparkContext
print(sc.master)        # yarn
print(sc.appName)       # Sales Analysis

# Read data using DataFrame API (via SparkSession)
df = spark.read.parquet("hdfs:///data/sales/2024/")

# Run SQL query
df.createOrReplaceTempView("sales")
result = spark.sql("""
    SELECT region, SUM(amount) as total
    FROM sales
    WHERE year = 2024
    GROUP BY region
    ORDER BY total DESC
""")
result.show()
```

Feature	SparkContext	SparkSession
Introduced in	Spark 1.0	Spark 2.0
Primary API	RDD	DataFrame / Dataset
SQL Support	Requires SQLContext	Built-in
Hive Support	Requires HiveContext	Via .enableHiveSupport()
Recommended	No (legacy)	Yes (modern standard)
Multiple per app	No (1 only)	Yes (multiple supported)

Driver Node

The Driver Node (or Driver Program) is the machine that runs the main() function of your Spark application. It is the central coordinator of your Spark job. The Driver is responsible for creating the SparkSession, building the DAG, coordinating with the Cluster Manager, and collecting final results.

Worker Node

A Worker Node is any machine in the cluster that can run Executor processes for a Spark application. Worker Nodes are the compute workhorses — they do the actual data processing. Each Worker Node can run one or more Executor processes, and each Executor runs multiple Tasks concurrently.

Executor Details

- An Executor is a JVM process launched on a Worker Node
- Each Executor has a fixed amount of CPU cores and memory assigned at startup
- Tasks within an Executor run in separate threads (one task per core)
- Executors cache data in memory to speed up iterative algorithms
- Executors communicate directly with the Driver to report task completion

Component	Runs On	Main Responsibility	How Many?
Driver	One designated machine	Coordinate the entire job	1 per Spark application
Cluster Manager	Master machine	Allocate cluster resources	1 per cluster
Worker Node	Any cluster machine	Host executors	Many (scales horizontally)
Executor	Worker nodes	Execute tasks, cache data	Multiple per Worker Node
Task	Inside Executor thread	Process one data partition	Many per Executor

4. Client Mode vs Cluster Mode

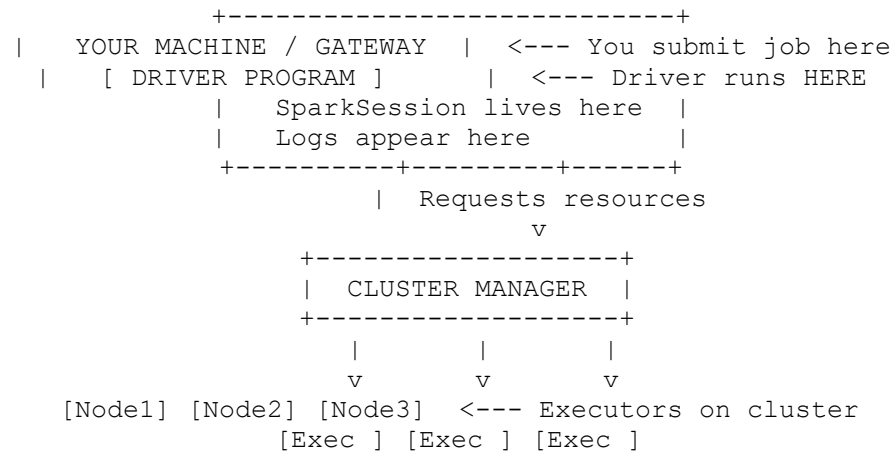
Overview

When you submit a Spark application to a cluster, you must decide where the Driver program runs. This choice is called the Deploy Mode, and there are two options: Client Mode and Cluster Mode. The mode you choose affects performance, stability, and how logs/output are collected.

Client Mode

In Client Mode, the Driver program runs on the machine where you submit the Spark job — typically your laptop, a gateway server, or a Jupyter notebook server. The Cluster Manager launches Executors on Worker Nodes, but the Driver stays on the submitting machine.

Client Mode Architecture



Characteristics of Client Mode

- Driver runs locally on the submitting machine
- If your local machine dies or loses network connection, the job FAILS
- Logs and print statements appear directly in your terminal
- Good for interactive development (Jupyter Notebooks, PySpark shell)
- Network traffic flows between Driver (local) and Executors (cluster) — can be slow if physically far apart

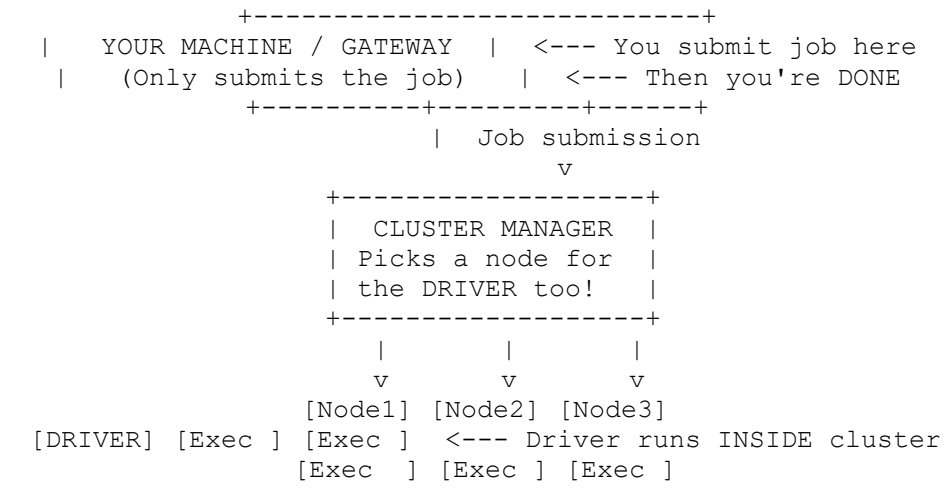
```
# Submit in CLIENT mode (default for spark-submit)
spark-submit \
  --master yarn \
```

```
  --deploy-mode client \       # Client mode (driver on your machine)
  --num-executors 10 \
  --executor-memory 4g \
  --executor-cores 2 \
  my_spark_app.py

# In PySpark shell (always client mode by default)
pyspark --master yarn

# In Python code
spark = SparkSession.builder \
    .master("yarn") \
    .config("spark.submit.deployMode", "client") \
    .getOrCreate()
```

4.3 Cluster Mode

In Cluster Mode, the Driver program runs inside the cluster — on one of the Worker Nodes (or a dedicated AM container in YARN). When you submit the job from your machine, your machine's role ends after submission. The Cluster Manager takes over and launches both the Driver and Executors inside the cluster.

Cluster Mode Architecture



Characteristics of Cluster Mode

- Driver runs inside the cluster on a Worker Node
- If your local machine dies or disconnects, the job CONTINUES (Driver is in the cluster)
- Logs are stored in the cluster; you must use yarn logs or Spark History Server to view them
- Lower network latency between Driver and Executors (all within same data center network)

- Best for production batch jobs and scheduled pipelines

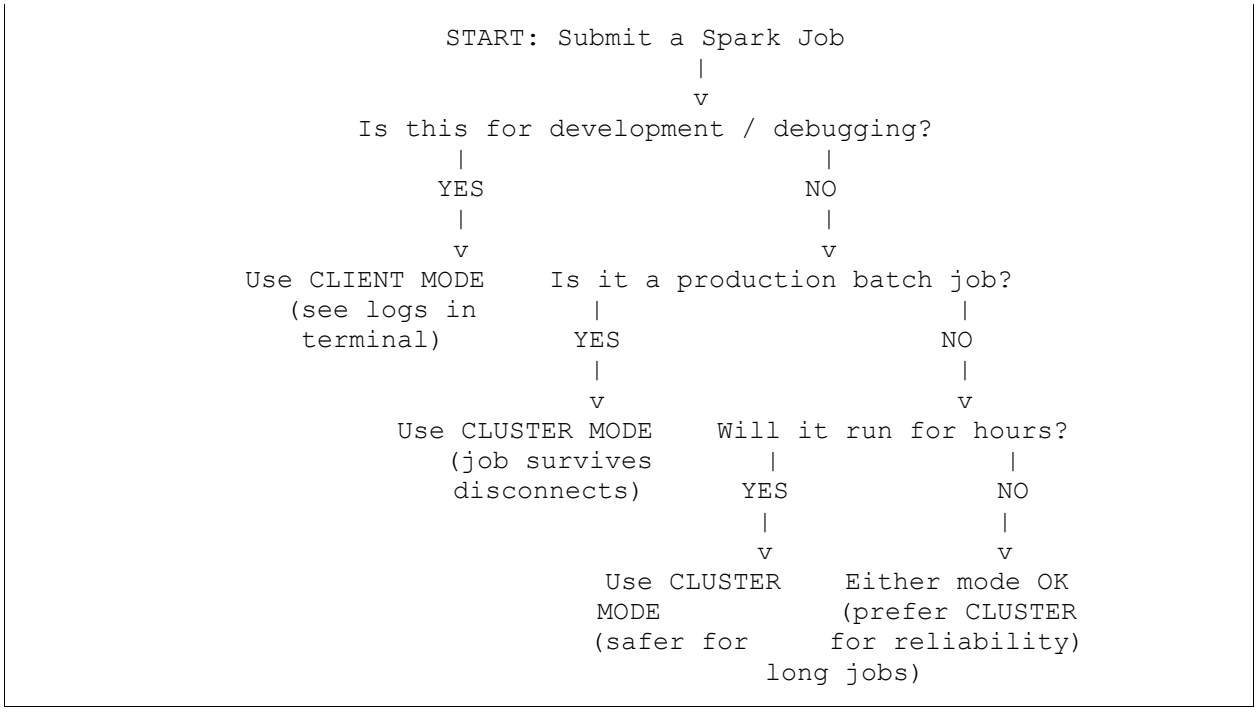
```
# Submit in CLUSTER mode (for production jobs)
spark-submit \
  --master yarn \
  --deploy-mode cluster \      # Cluster mode (driver inside cluster)
  --num-executors 20 \
  --executor-memory 8g \
  --executor-cores 4 \
  --driver-memory 4g \
  my_spark_app.py

# After submission, your terminal shows only:
#   "Application submitted. Tracking URL: http://..."
# Use this to view logs:
yarn logs -applicationId application_1234567890_0001
```

4.4 Side-by-Side Comparison

Feature	Client Mode	Cluster Mode
Driver Location	On submitting machine (your laptop/gateway)	Inside the cluster (on a Worker Node)
Job dependency on submitter	YES - job fails if local machine disconnects	NO - job runs independently in cluster
Logs / Output	Directly in your terminal	Stored in cluster (use yarn logs)
Network efficiency	Lower - Driver-Executor traffic crosses network boundary	Higher - all within cluster network
Startup time	Faster - no Driver container needed in cluster	Slightly slower - cluster allocates Driver container
Use case	Interactive dev, Jupyter, debugging	Production jobs, scheduled pipelines, long-running jobs
PySpark shell	Always client mode	Not supported (interactive needs local Driver)
Supported managers	YARN, Mesos, Kubernetes, Standalone	YARN, Mesos, Kubernetes, Standalone

4.5 How to Choose: Decision Guide

Choosing Between Client Mode and Cluster Mode



Golden Rule

Use Client Mode when you want to see results interactively (notebooks, shell, testing). Use Cluster Mode for all production pipelines and scheduled jobs where you need fault tolerance and reliability.

4.6 Local Mode (Bonus)

Local Mode is a special mode where both the Driver and Executor run on your single machine without any cluster. It is used exclusively for development and testing. You specify local[N] where N is the number of CPU threads to use, or local[*] to use all available cores.

```
# Local modes
spark = SparkSession.builder.master("local").getOrCreate()   # 1 thread
(sequential)
spark = SparkSession.builder.master("local[4]").getOrCreate() # 4 threads
spark = SparkSession.builder.master("local[*]").getOrCreate() # all CPU
cores

# Local mode is the default when you just do:
pyspark   # starts PySpark in local[*] mode
```

Mode	Driver Location	Executor Location	Use Case

local	Your machine	Your machine (same JVM)	Single-thread development
local[N]	Your machine	Your machine (N threads)	Multi-thread local testing
yarn --deploy-mode client	Your machine	YARN cluster nodes	Interactive development on cluster
yarn --deploy-mode cluster	YARN cluster node	YARN cluster nodes	Production jobs

Summary & Quick Reference

Topic	Key Takeaway
Big Data	Data too large for single-machine tools; needs distributed processing
Apache Spark	Distributed in-memory processing engine, up to 100x faster than MapReduce
PySpark	Python API for Spark using Py4J bridge to communicate with JVM
RDD	Immutable, distributed, fault-tolerant collection; the core data abstraction
DAG	Logical execution plan Spark builds before running; enables optimization
SparkContext	Legacy entry point (Spark 1.x); creates RDDs; 1 per app
SparkSession	Modern unified entry point (Spark 2.x+); recommended for all new code
Cluster Manager	Manages cluster resources (YARN, Kubernetes, Standalone, Mesos)
Driver Node	Runs main(), coordinates the job, hosts SparkSession/SparkContext
Worker Node	Machine in cluster that hosts Executors; does actual data processing
Executor	JVM process on Worker; runs Tasks, caches data
Client Mode	Driver on submitting machine; good for dev/debugging; job fails if disconnected
Cluster Mode	Driver inside cluster; job survives disconnection; best for production